

CS 6160: Introduction to Cryptology

Programming Assignment 2

Shibam Ghosh
IIT Hyderabad

September 3, 2026

Please provide proper instructions on how to run the program. The submission deadline is 22nd September, 2026.

1 AES-128 Software Implementation vs. AES-NI (70 Points)

1.1 Objective

The objective of this assignment is to compare a conventional software implementation of an AES-like block cipher with an implementation based on AES-NI instructions. You will implement both encryption and decryption in the following two ways:

1. A conventional software implementation.
2. An implementation using AES-NI instructions.

You will then benchmark the implementations and compare their performance.

Cipher Specification and Round Keys

In this assignment, you are **not required to implement the standard AES key schedule**. Let the given AES-128 master key be denoted by K . You should use this key directly as the pre-whitening key:

$$k_0 = K.$$

The round keys are generated recursively as follows. For $i = 1, \dots, 10$,

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i,$$

where RC_i denotes the i -th round constant represented as a 128-bit value, given in the following table:

Round	1	2	3	4	5	6	7	8	9	10
$Rcon$	01	02	04	08	10	20	40	80	1B	36

Thus, your implementation uses the following sequence of keys:

$$k_{-1}, k_0, k_1, \dots, k_9.$$

You must use exactly the same round-key generation procedure in both your software and AES-NI implementations.

1.2 Part 1: Conventional Software Implementation

Implement the encryption and decryption algorithms in C or C++ without using AES-NI instructions. Your implementation should explicitly implement the relevant AES transformations. The round keys must be generated using the key-generation rule specified above.

Part 2: AES-NI Implementation

Implement the same encryption and decryption algorithms using AES-NI instructions. You may use compiler intrinsics provided by the compiler, for example through:

```
#include <immintrin.h>
```

Typical AES-NI intrinsics include encryption instructions such as:

```
_mm_aesenc_si128  
_mm_aesenclast_si128
```

and the corresponding decryption instructions such as:

```
_mm_aesdec_si128  
_mm_aesdeclast_si128
```

Important: Your AES-NI implementation must implement exactly the same cipher and use exactly the same round keys as your conventional software implementation. Therefore, for the same key and plaintext, both implementations must produce identical ciphertexts. Similarly, for the same key and ciphertext, both implementations must recover identical plaintexts.

Correctness Testing

Before performing any performance measurements, verify the correctness:

1. Your encryption and decryption implementations are correct and it correctly decrypts.
2. AES-NI encryption and decryption implementations are correct and it correctly decrypts.
3. AES-NI encryption and your implementation produce identical ciphertexts.
4. AES-NI decryption and your implementation produce identical plaintexts.

To test these, you will generate random plaintexts and keys. For a randomly generated plaintext P and key K , verify the following properties:

$$D_{\text{SW}}(E_{\text{SW}}(P)) = P,$$

$$D_{\text{AESNI}}(E_{\text{AESNI}}(P)) = P,$$

and

$$E_{\text{SW}}(P) = E_{\text{AESNI}}(P).$$

Performance Benchmark

Benchmark the following four operations separately:

1. Software encryption.
2. AES-NI encryption.
3. Software decryption.
4. AES-NI decryption.

Use a sufficiently large input size so that the timing measurements are meaningful. For example, you may benchmark inputs of size:

- 1 KB,
- 2 KB,
- 4 KB,
- 32 KB,
- 64 KB.

The input should consist of a large number of independent 16-byte blocks.

Measuring Execution Time

Measure the total execution time required to process the entire input. If N bytes are processed in T seconds, report the throughput as

$$\text{Throughput} = \frac{N}{T}.$$

You may report the result in KB/s. Perform the benchmark multiple times and report either:

- the average execution time, or
- the median execution time.

Clearly state which method you use.

Measuring Cycles per Byte

In addition to execution time, estimate the number of CPU cycles required to process one byte. Let:

- C_{start} be the cycle counter before processing the input;
- C_{end} be the cycle counter after processing the input;
- N be the total number of bytes processed.

The total number of cycles is

$$C = C_{\text{end}} - C_{\text{start}}.$$

The cycles per byte (CPB) is then

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}.$$

For example, if encrypting $16N_b$ bytes consisting of N_b AES blocks requires C cycles, then

$$\text{CPB} = \frac{C}{16N_b}.$$

Recommended Cycle Counter

On x86 processors, you may use the timestamp counter instruction RDTSC through compiler intrinsics. For example:

```
#include <x86intrin.h>

unsigned long long start = __rdtsc();

/* encryption or decryption */

unsigned long long end = __rdtsc();
```

Then compute

```
cycles = end - start;
cycles_per_byte = (double) cycles / total_bytes;
```

Note: The timestamp counter is sensitive to effects such as instruction reordering, caching, processor frequency scaling, and operating-system scheduling. For more reliable measurements, you may use serialization instructions or the appropriate serialized timestamp-counter routines supported by your compiler and processor.

Benchmarking Recommendations

To obtain meaningful performance measurements:

- Do not include key generation or initialization in the main encryption/decryption timing loop.
- Generate the round keys before starting the measurement.
- Process a sufficiently large amount of data.
- Repeat each experiment multiple times.
- Ensure that the compiler does not optimize away the encryption or decryption computation.
- Use an optimized compilation level such as `-O2` or `-O3`.

For the AES-NI implementation, ensure that the compiler is allowed to generate AES instructions. Depending on your compiler and platform, this may require an option such as:

`-maes`

and, if required,

`-msse2`

or the corresponding architecture-specific options.

Expected Results

Prepare a table similar to the following.

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Throughput (MB/s)					
Software Encryption					
AES-NI Encryption					
Software Decryption					
AES-NI Decryption					
Cycles per Byte (CPB)					
Software Encryption					
AES-NI Encryption					
Software Decryption					
AES-NI Decryption					

Questions for Discussion

In your report, briefly discuss the following questions:

1. How much faster is AES-NI encryption compared with your software implementation?
2. How much faster is AES-NI decryption compared with your software implementation?
3. Is the performance improvement similar for encryption and decryption? If not, why might they differ?
4. Compare the throughput and cycles per byte obtained for encryption and decryption.
5. How do the measured results change as the message size increases?

Submission

Submit the following:

1. Your complete C or C++ source code.
2. Instructions for compiling and running your program.
3. A short report containing:
 - the machine and processor used;
 - compiler and compiler flags;
 - throughput measurements;
 - cycles-per-byte measurements;
 - and a brief discussion of the results.

2 Multiplication Table of an Extension Field (30 Points)

Write a C or C++ program to generate the multiplication table of the extension field

$$GF(2^3) = \mathbb{F}_2[x]/(P(x)),$$

where the irreducible polynomial is

$$P(x) = x^3 + x + 1.$$

The multiplication table is an 8×8 table, since $GF(2^3)$ contains $2^3 = 8$ elements. Your program should:

1. Represent each element of $GF(2^3)$ using 3 bits, corresponding to

$$a_2x^2 + a_1x + a_0, \quad a_0, a_1, a_2 \in \{0, 1\}.$$

2. Implement multiplication of two field elements by:

- multiplying the corresponding polynomials over $\mathbb{F}_2$; and
- reducing the resulting polynomial modulo

$$P(x) = x^3 + x + 1.$$

3. Generate and print the complete 8×8 multiplication table.
4. Clearly label the rows and columns using the corresponding field elements, which should look something like this.

$\times$	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

Hints

Use the following representation for the elements:

Binary	Polynomial
000	0
001	1
010	x
011	$x + 1$
100	x^2
101	$x^2 + 1$
110	$x^2 + x$
111	$x^2 + x + 1$

For example,

$$x \cdot x^2 = x^3 \equiv x + 1 \pmod{x^3 + x + 1}.$$

Hence, using the above representation,

$$010 \times 100 = 011.$$